Road Lane Detection & Tracking from Car Dashcam Video

Using Image Processing and Deep Learning

การตรวจจับและติดตามเส้นแบ่งช่องจราจร
จากวิดีโอกล้องติดตามรถยนต์ด้วยการประมวลผลภาพและการเรียนรู้เชิงลึก



จัดทำโดย
650710051 นางสาวจิราวรรณ อยู่จงดี
660710095 นายมโนรินทร์ นันทะนิ

คำถามจาก Midway

1. โครงงานนี้เป็นวิจัยหรือแอปพลิเคชันและจะได้เห็นอะไรในการสอบจบ

หลังจากปรึกษากันแล้ว ทีมได้ข้อสรุปว่าจะทำโครงงานนี้ในรูปแบบงานวิจัย ผลลัพธ์เป็นระบบต้นแบบที่นำภาพถนนที่เก็บไว้มาประมวลผลทีละภาพจนครบทั้งชุด ยังไม่ได้ประมวลผลภาพจากกล้องแบบทันที และยังไม่ทำเป็นแอปพลิเคชัน เนื่องจากจุดเน้นหลักคือประสิทธิภาพการตรวจจับเส้นเลนบนถนนไทย ซึ่งในสภาวะกลางคืนและทางโค้งยังได้ค่าต่ำ ทีมจึงเลือกพัฒนาโมเดลและข้อมูลก่อน

คำถามจาก Midway

1. โครงงานนี้เป็นวิจัยหรือแอปพลิเคชันและจะได้เห็นอะไรในการสอบจบ

- ชุดข้อมูลถนนในไทยที่ Label แล้ว
- โมเดลตรวจจับเส้นเลนที่ปรับแต่งสำหรับถนนในไทย
- ภาพผลลัพธ์ระบายพื้นที่เลน
- ค่าประเมินวัดประสิทธิภาพของโมเดล
- ข้อค้นพบจากการทดลอง

คำถามจาก Midway

2. ตัดท้องฟ้าอย่างไร

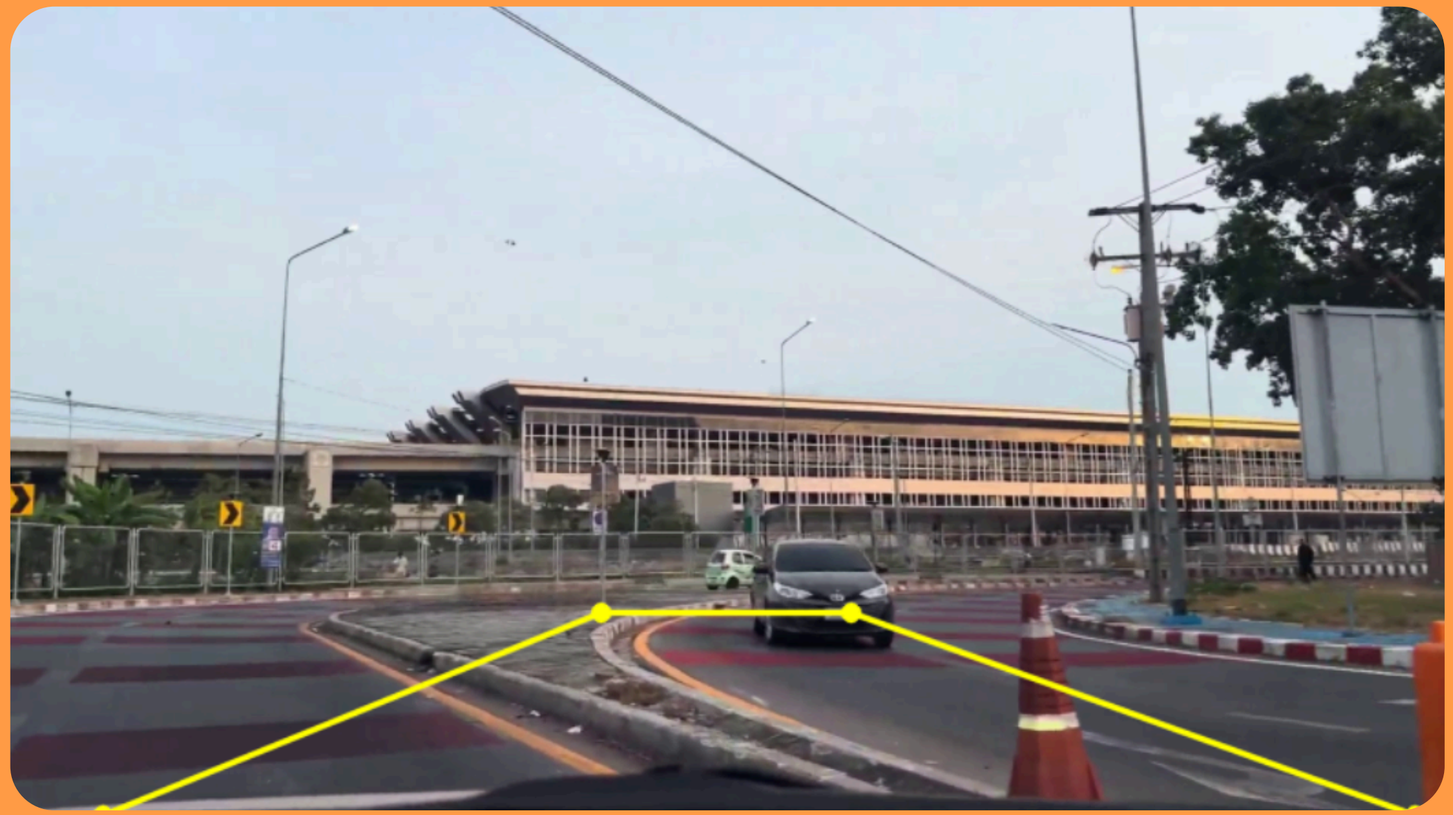
ในช่วงแรก เรากำหนดบริเวณที่สนใจ (ROI) โดยปรับค่าเองจากการทดลองกับภาพตัวอย่าง แล้วใช้ค่าเดียวกันนั้นกับทุกภาพ แต่เมื่อตรวจดูผลด้วยตาทีละภาพ พบว่าตำแหน่งของเส้นเลนในแต่ละภาพไม่เหมือนกัน บางภาพเส้นเลนเอียง หรืออยู่คนละตำแหน่งกับภาพที่ใช้ปรับค่า เช่น ทางโค้ง ทางขึ้นลงเนิน หรือรถไม่ได้วิ่งตรงกลางเลน ทำให้ ROI ตัดเส้นเลนที่รถวิ่งอยู่ออกไปบางส่วน หรือตัดออกไปทั้งเส้น เราจึงเห็นว่าค่า ROI ชุดเดียวใช้กับทุกภาพไม่ได้ จึงเปลี่ยนมาให้โมเดลที่ฝึกมาแล้วเป็นผู้ทำนายเส้นเลนจากภาพเต็มแทน

ตัวอย่างภาพที่ใช้การจำกัดพื้นที่ที่สนใจ (ROI)



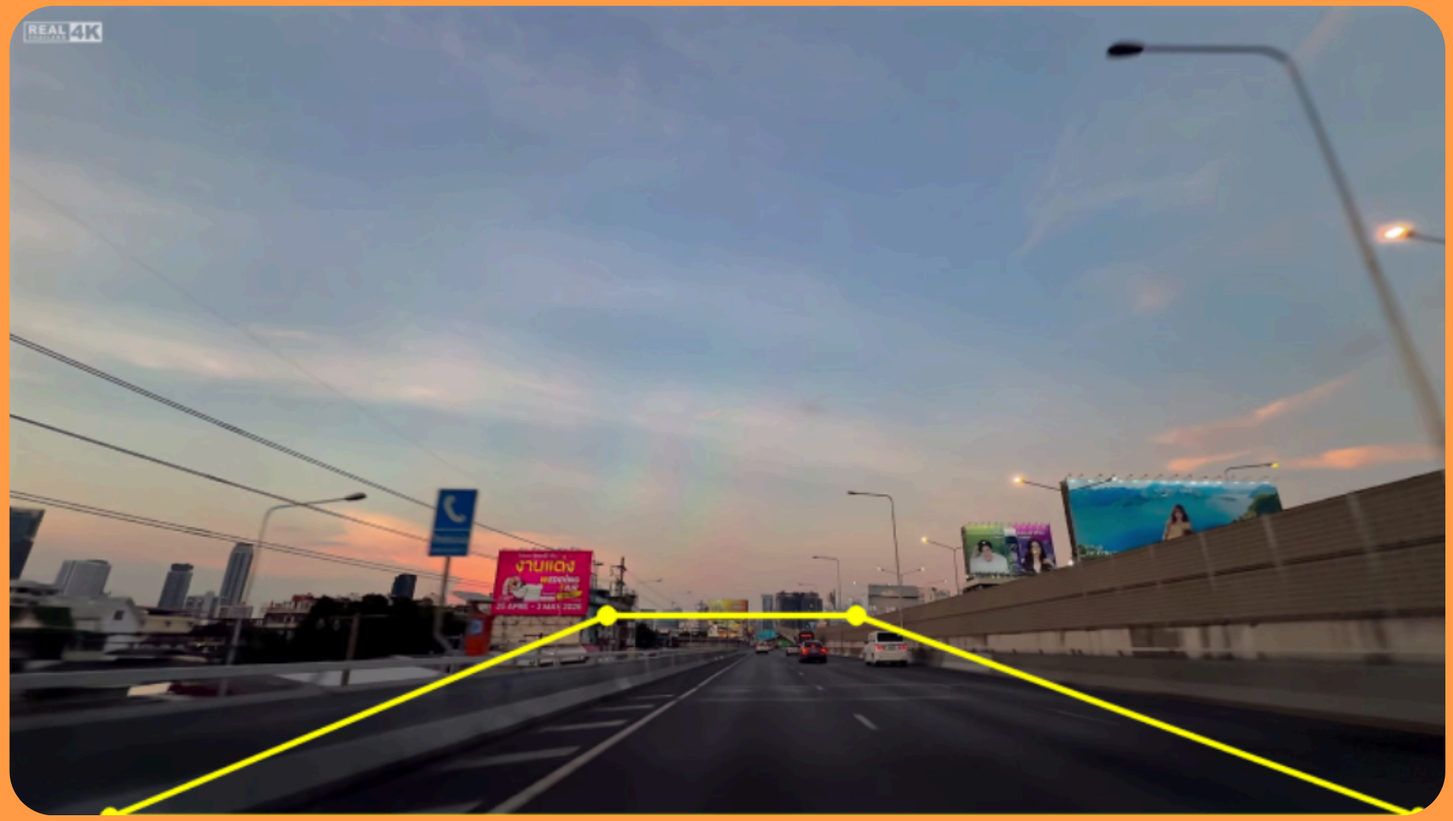
เส้นเลนอยู่ในพื้นที่ที่สนใจ

ตัวอย่างภาพที่ใช้การจำกัดพื้นที่ที่สนใจ (ROI)



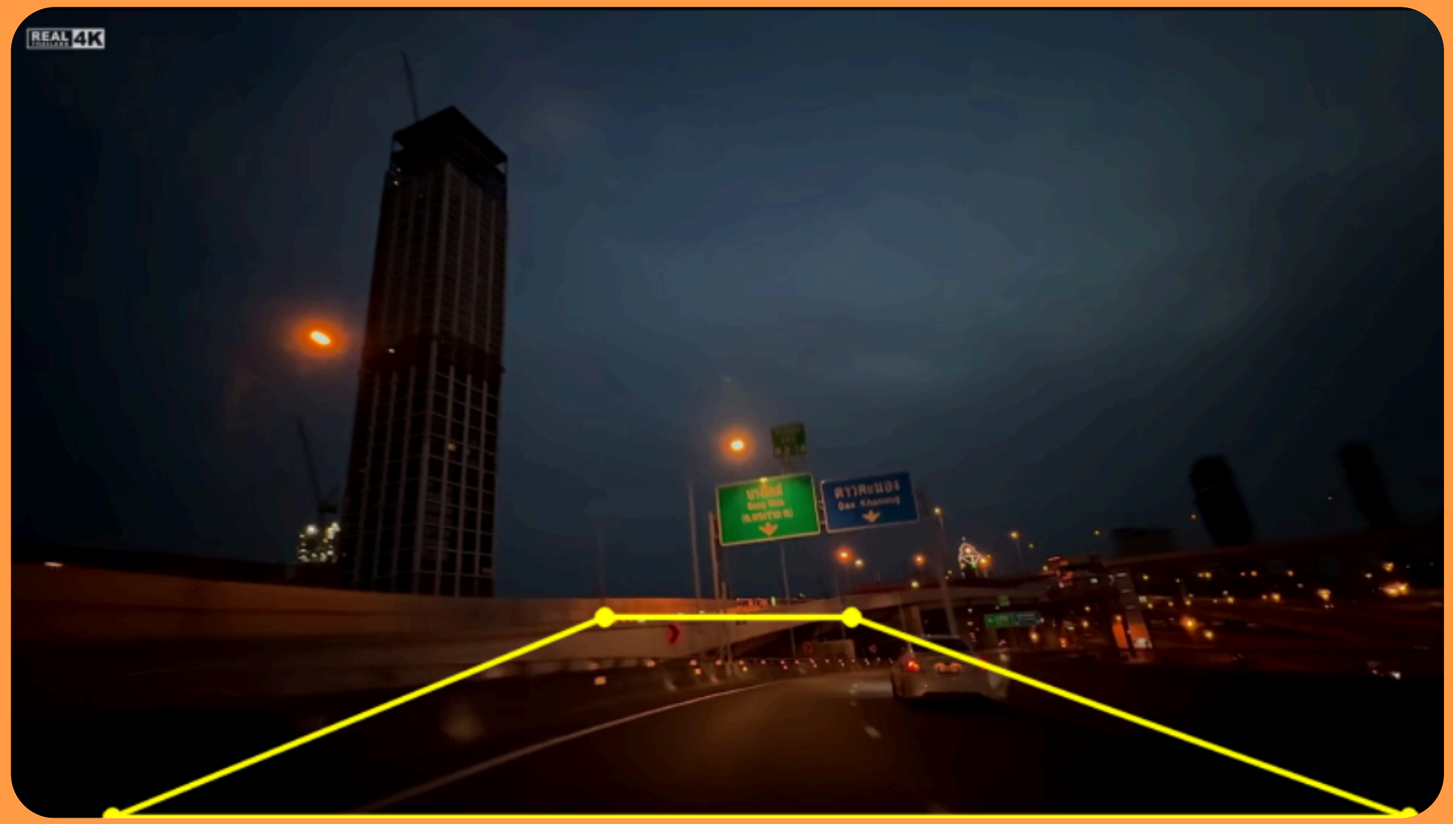
บนทางโค้ง เส้นเลนในภาพจะเอียงหรือเลื่อนไปด้านข้าง แต่ ROI ยังอยู่ตำแหน่งเดิม จึงไปครอบเลนข้าง ๆ แทนที่จะเป็นเลนที่รถวิ่งอยู่

ตัวอย่างภาพที่ใช้การจำกัดพื้นที่ที่สนใจ (ROI)



ยังมีท้องฟ้าและ Noise อื่นๆในพื้นที่ที่สนใจ

ตัวอย่างภาพที่ใช้การจำกัดพื้นที่ที่สนใจ (ROI)



ยังมีท้องฟ้าและ Noise อื่นๆในพื้นที่ที่สนใจ

คำถามจาก Midway

3. ด้านการแบ่ง Dataset แบ่งอย่างไร เพื่อให้โมเดลเห็นข้อมูลแบบกระจาย

Random Split (การสุ่มระดับภาพเดียว) → จะเกิด เฉลยรั่ว
เพราะจะทำให้โมเดลนั้นจำฉากเดิมได้และจะทำให้ค่าคะแนนสูงเกินจริงเกินไป

EX. CLIP 01 (คลิปเดียวกัน)

Frame 1 Train	Frame 2 Train	Frame 3 TEST!!	Frame 4 Train

คำถามจาก Midway

วิธีแก้: Group (โดยกำหนด Clip ID)

หลักการคือทำงานแบบแยกตามคลิป
กฎเหล็ก: ภาพทั้งหมดที่สกัดมาจากคลิปกัน “ต้องอยู่ในชุดเดียวกันเท่านั้น”
ห้ามแยกภาพข้ามชุดโดยเด็ดขาด → ทำให้ Test set เป็นภาพจากถนนและฉากที่ไม่เคยเจอมาก่อนเลย

EX.

Clip 01 (ทั้งคลิป 25 ภาพ)
สภาพแวดล้อม : ทางตรง (Normal)

Train Set

ทุกเฟรมในคลิปอยู่ใน Train

CLIP 02 (ทั้งคลิป 20 ภาพ)
สภาพแวดล้อม : ทางโค้ง (Curve)

Test Set

ทุกเฟรมในคลิปอยู่ใน Test Set

คำถามจาก Midway

อัลกอริทึม Group Stratified Split :

- 1.) จัดหมวดหมู่คลิปตามสภาวะ : ทางตรง(Normal), ทางโค้ง(Curve), กลางคืน(night)
- 2.) กระจายคลิปในแต่ละหมวด ให้มีสัดส่วน Train, Validation, Test ให้เท่าๆกัน
- 3.) ควบคุมอัตราส่วนรวมให้อยู่ในกรอบเป้าหมาย 70% : 15% : 15%

Train	Validation	Test
70%	15%	15%

คำถามจาก Midway

4. Tech Stack / Library และ AI ที่ใช้

PyTorch

แกนหลักเทรนโมเดล, torch.amp , DataLoader

```
model.train()  # 1. เปิดโหมดเทรน (เปิด BatchNorm / Dropout)

    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)

        # 2. ล้าง Gradient เดิม (ห้ามลืม ไม่งั้น Gradient สะสมข้ามรอบ)
        optimizer.zero_grad()

        # 3. Forward Pass (ยิงภาพเข้าโมเดล ได้ผลทายออกมา)
        outputs = model(images)

        # 4. Compute Loss (วัดความผิดพลาดเทียบกับเฉลย)
        loss = criterion(outputs, masks)  # เช่น BCE + Dice Loss

        # 5. Backward Pass (คำนวณ Gradient ทุกสาดับชั้นด้วย Chain Rule)
        loss.backward()

        # 6. Optimizer Step (ปรับค่าน้ำหนัก W จริงตาม Learning Rate)
        optimizer.step()
```

Segmentation Models

ดึงสถาปัตยกรรมโมเดลและพรีเทรนเวท

คำถามจาก Midway

Tech Stack / Library และ AI ที่ ใช้

Architectures / Model

- DeepLabV3+
- U-Net

Encoders

- ResNet-34,
- ResNet-50

Pretrained Weights

- ImageNet
- SSL



จัดการ Array, พิกัดเส้นเลน, อ่าน/แบ่งไฟล์ data(.csv), ฟิตเส้นโค้งพหุนาม



โหลดภาพ ปรับขนาด วาด mask และ คำนวณเส้นโครงร่าง ทำ Augmentation หา Contour วาดและระบายพื้นที่เลน



แสดงและบันทึกภาพผลลัพธ์

คำถามจาก Midway

Tech Stack / Library และ AI ที่ใช้

GroupShuffleSplit / GroupKFold แบ่งข้อมูล ป้องกัน clip leakage

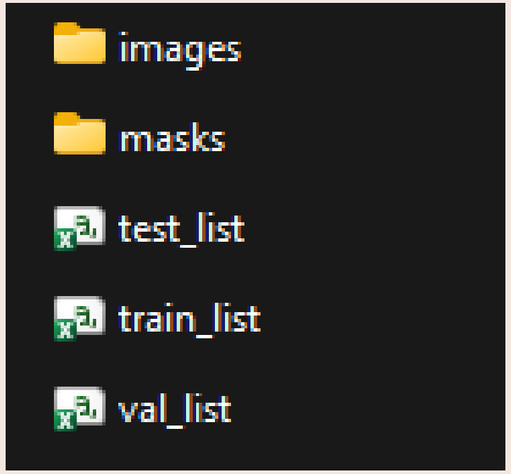
```python
from sklearn.model_selection import StratifiedGroupKFold

    # 1. กำหนดการแบ่ง: Group ตาม clip_id และ Stratify สัดส่วน category
    sgkf = StratifiedGroupKFold(n_splits=20, shuffle=True,
  random_state=42)

    for fold, (_, val_idx) in enumerate(sgkf.split(df, y=df["category"],
  groups=df["clip_id"])):
        df.loc[val_idx, "fold"] = fold

    # 2. ตัดสัดส่วน 70% Train / 15% Val / 15% Test
    train_df = df[df["fold"] < 14]
    val_df   = df[(df["fold"] >= 14) & (df["fold"] < 17)]
    test_df  = df[df["fold"] >= 17]

    # 3. ยืนยันทางสถิติ: คลิปต้องไม่ซ้ำกันข้ามชุดเด็ดขาด (0% Leakage)
    assert len(set(train_df["clip_id"]) & set(test_df["clip_id"])) == 0
```



- Group by clip_id: เฟรมจากวิดีโอคลิปเดียวกัน ต้องอยู่ชุดเดียวกัน 100% (ป้องกัน Data Leakage)
- Stratify by category: คุมสัดส่วน Normal, Curve, Night ให้สมดุลทุกชุด
- Split 70:15:15: แบ่งผ่าน 20 Folds ส่งออกเป็น train_list, val_list, test_list (.csv)

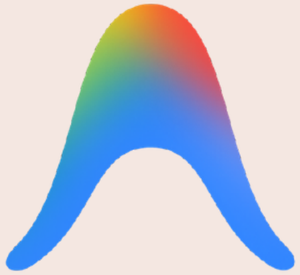
ใช้ **StratifiedGroupKFold** :

1. Group ตาม clip_id เพื่อขังคลิปเดียวกันไว้ในชุดเดียว
2. Stratify ตามสภาพถนน เพื่อกระจายสัดส่วนให้สมดุล
3. ตัดสัดส่วน Train 70 : Val 15 : Test 15

คำถามจาก Midway

Tech Stack / Library และ AI ที่ ใช้

AI



Antigravity CLI : โมเดลประมวลผลคำสั่ง รันคำนวณสถิติ และจัดการโค้ด



- **Claude :** โมเดลวิเคราะห์เชิงลึก วางโครงสร้างสถาปัตยกรรมโมเดล ออกแบบการทดลอง และแก้ปัญหาทางสถิติ

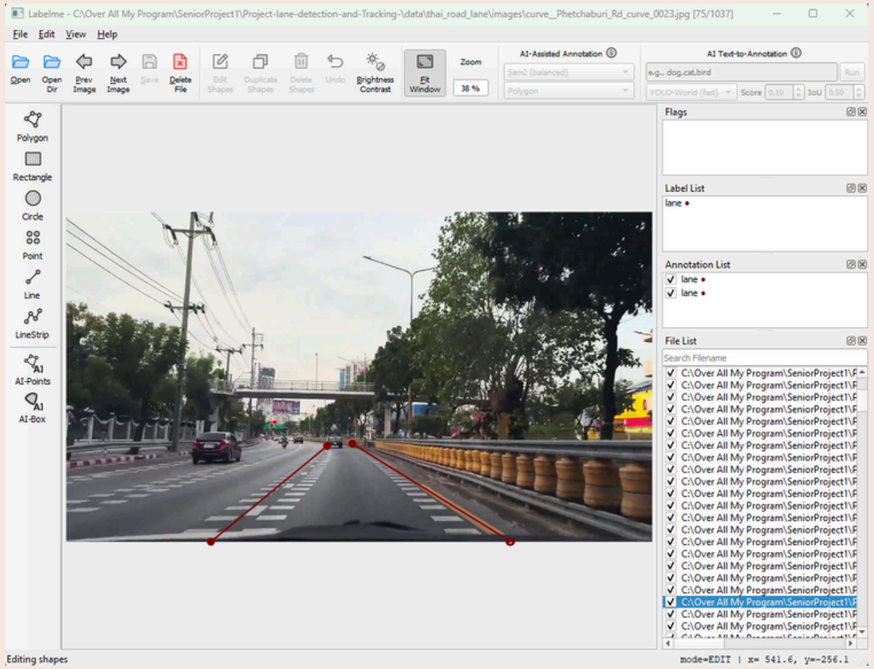
คำถามจาก Midway

5. ทำชุดทดสอบ Label อย่างไร

ในส่วนที่ทำ Label จะเป็นถนนประเทศไทย

เครื่องมือและรูปแบบการวาด (Annotation Tool)

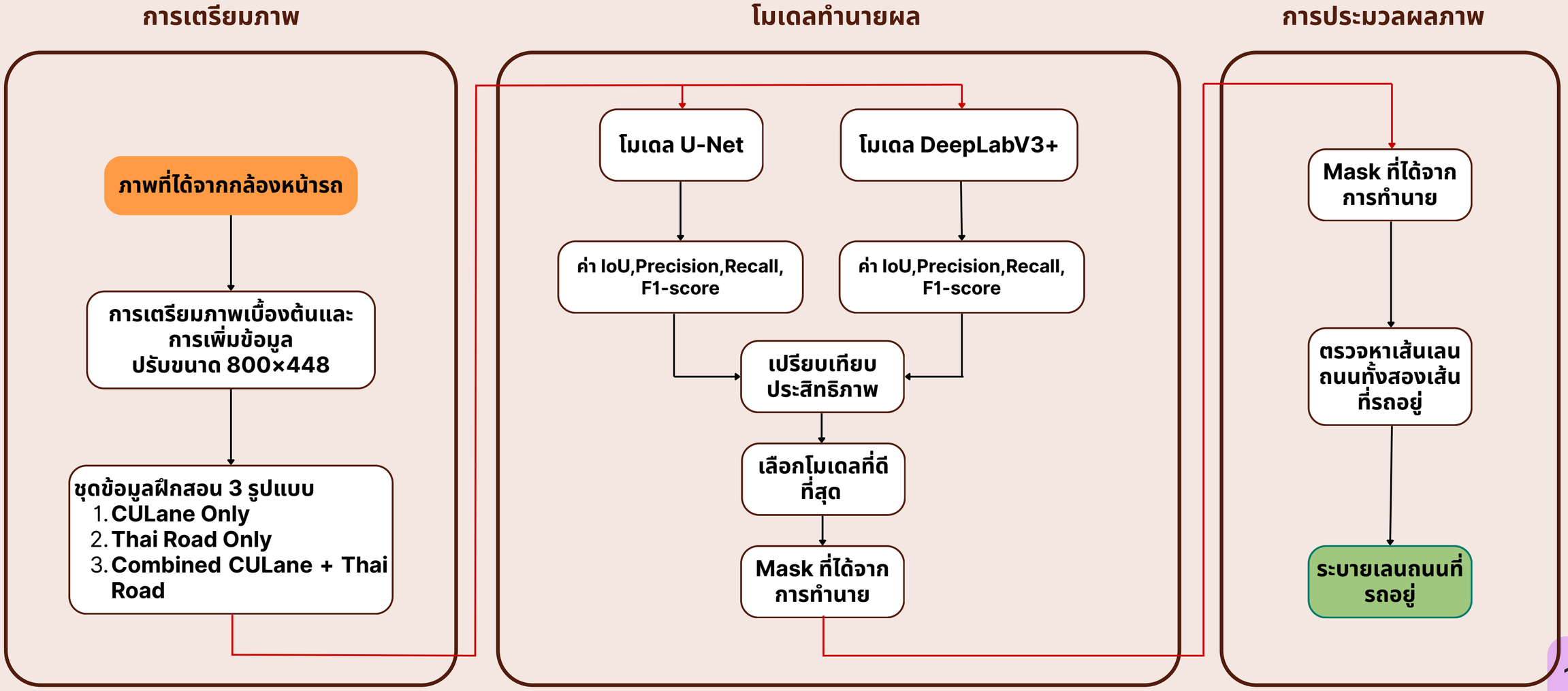
- เครื่องมือ:ใช้โปรแกรม Labelme
- ชื่อคลาส (Label): กำหนดชื่อเดียวคือ `lane`
- คลิกจุดตามกึ่งกลางแนวเส้นแบ่งช่องจราจร (Ego-lane boundaries) ทั้งฝั่งซ้ายและฝั่งขวาของรถ (2 เส้นต่อภาพ)
- กรณีเส้นประ หรือเส้นลบเลือน ให้ลากเชื่อมตามแนวต่อเนื่องของเส้นทาง
- บันทึกผลออกมาเป็นไฟล์ `.json` ต่อ 1 ภาพ (เช่น `normal__clip01_00000.json`)



หน้าโปรแกรม LabelMe



โครงสร้างสถาปัตยกรรมของระบบ



1. การเตรียมภาพ

รายละเอียดของ Data set

ในส่วนของCULane



55,426 รูป
ภาพทางด่วน / ทางตรงกลางวันปกติ

16,210 รูป
ในเมือง / ทางโค้งในซอย / แดดสะท้อนจ้า

15,710 รูป
กลางคืน / แสงน้อย / เงานสะพาน

รวม = 87,346 รูป

โดยภาพทั้งทั้งหมด 87,357 ภาพ นับเป็น footage ทั้งหมด 744 วิดีโอ โดยแบ่งเป็น

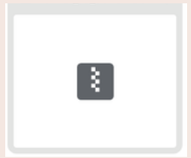
- ทางตรงปกติ = 343 Video
- ทางโค้ง = 302 Video
- ทางตอนกลางคืน = 99 Video

และในกลุ่มที่มีภาพตอนกลางคืนแฝงอยู่ในคือภาพ Footage ทางโค้งที่มีการแฝงอยู่ถึง 109 Video ถ้าเป็นภาพทั้งหมดนับเป็น 5,960 ภาพ

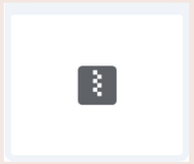
1. การเตรียมภาพ

รายละเอียดของ Data set

ในส่วนของถนนในประเทศไทย



662 รูป
ภาพทางด่วน / ทางตรงกลางวันปกติ



252 รูป
ทางโค้ง



123 รูป
กลางคืน

รวม = 1,037 รูป

โดยภาพทั้งทั้งหมด 1,037 ภาพ นับเป็น footage ทั้งหมด 54 วิดีโอ โดยแบ่งเป็น

- ทางตรงปกติ = 33 Video
- ทางโค้ง = 17 Video
- ทางตอนกลางคืน = 4 Video

และในกลุ่มที่มีภาพตอนกลางคืนแฝงอยู่ในคือภาพ Footage ทางโค้งที่มีการแฝงอยู่ถึง 3 Video ถ้าเป็นถาพทั้งหมดนับเป็น 85 ภาพ

1. การเตรียมภาพ

การทำ Augmentation

สุ่มพลิกภาพซ้าย–ขวา และสุ่มปรับความสว่าง เฉพาะตอนฝึกโมเดล เพื่อให้ชุดข้อมูลมีความหลากหลาย เพื่อจำลองทางโค้งทั้งสองทิศและสภาพแสงที่ต่างกัน ทำให้โมเดลเรียนรู้ได้หลากหลายขึ้น

```
h, w = IMG_SIZE
            image = cv2.resize(image, (w, h), interpolation=cv2.INTER_LINEAR)
            mask = cv2.resize(mask, (w, h), interpolation=cv2.INTER_NEAREST)

            if self.augment:
                image, mask = self._augment(image, mask)

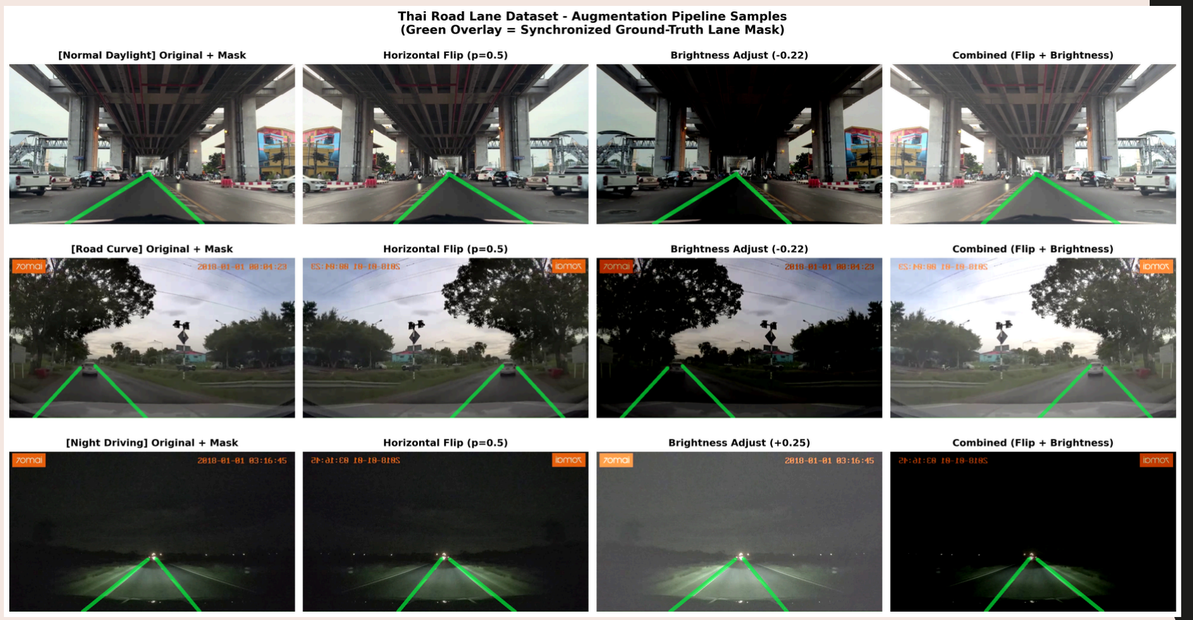
            # แปลงเป็น Tensor (shape: C, H, W)
            image_t = torch.from_numpy(image).float().permute(2, 0, 1) / 255.0
            mask_t = (torch.from_numpy(mask).float().unsqueeze(0) > 127.0).float()

            return image_t, mask_t

        def _augment(self, image, mask):
            # สุ่มพลิกซ้าย-ขวา ทั้งภาพและ Mask พร้อมกัน (50%)
            if np.random.rand() < 0.5:
                image = cv2.flip(image, 1)
                mask = cv2.flip(mask, 1)

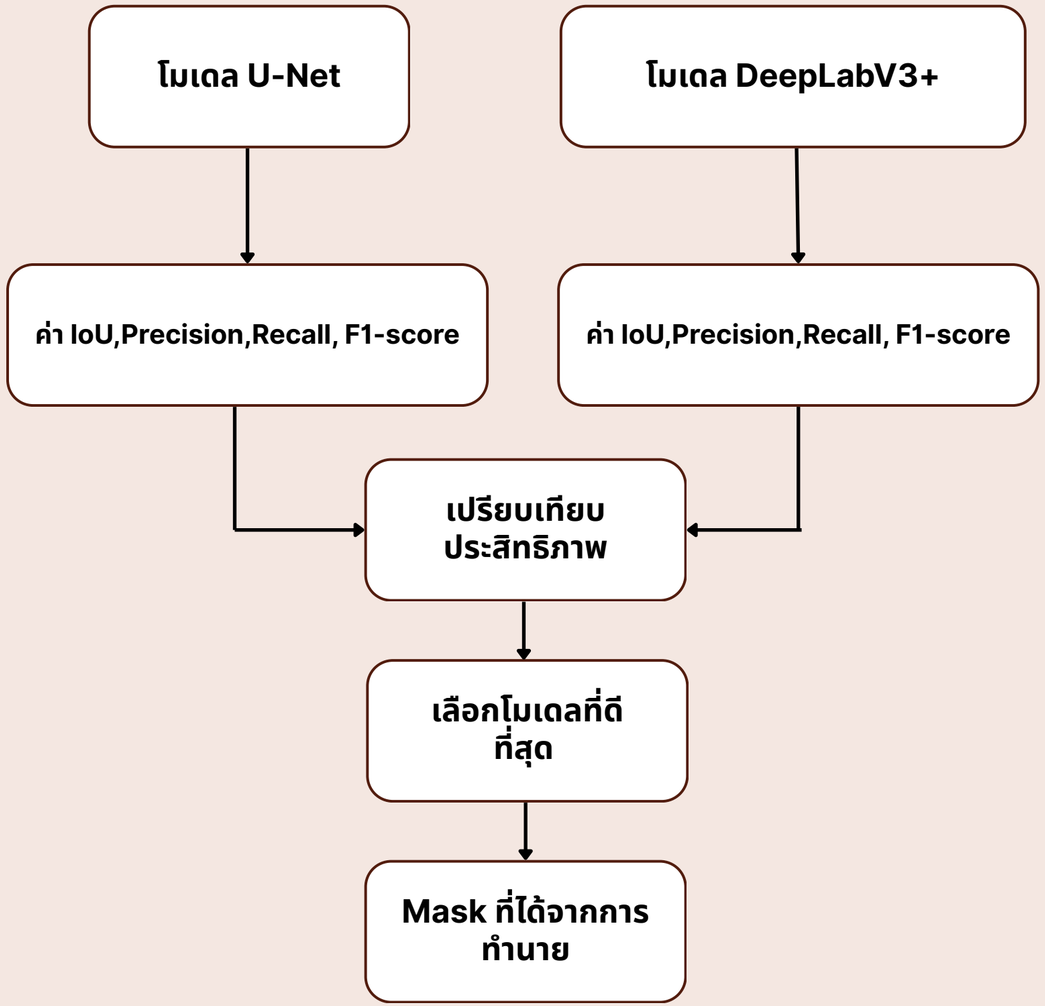
            # สุ่มปรับความสว่าง +-30% (30%)
            if np.random.rand() < 0.3:
                delta = np.random.uniform(-0.3, 0.3) * 255.0
                image = np.clip(image.astype(np.float32) + delta, 0, 255).astype(np.uint8)

            return image, mask
```



2. โมเดลทำนายผล

แผนภาพการไหลของข้อมูลและกระบวนการทำงานรวม (System Pipeline)



Model ที่เลือกใช้

Unet

Encoder = resnet34, resnet50
Pretrained Weights = imagenet, ssl

DeeplabV3+

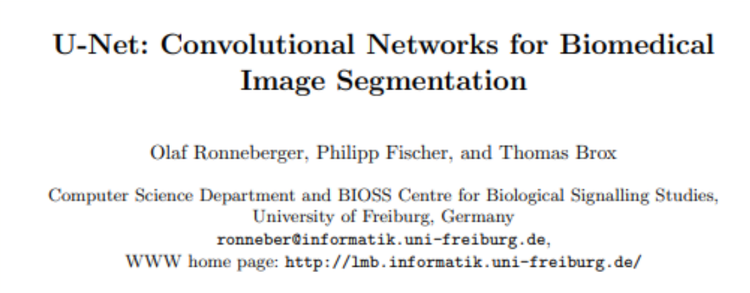
Encoder = resnet34, resnet50
Pretrained Weights = imagenet, ssl

ได้เพิ่มEpochs จาก 20 ไปเป็น 30

```
ENCODER_NAME = "resnet34"        # ตัวเลือก: resnet34, resnet50
ENCODER_WEIGHTS = "imagenet"     # ตัวเลือก: imagenet, ssl (เฉพาะ resnet50)
EPOCHS = 30
BATCH_SIZE = 16
LEARNING_RATE = 1e-4
DATASET_ROOT = "/home/jovyan/shared/donut/CarDashCam/CULane"
```

เหตุผลที่ใช้ Unet ในการทดลอง

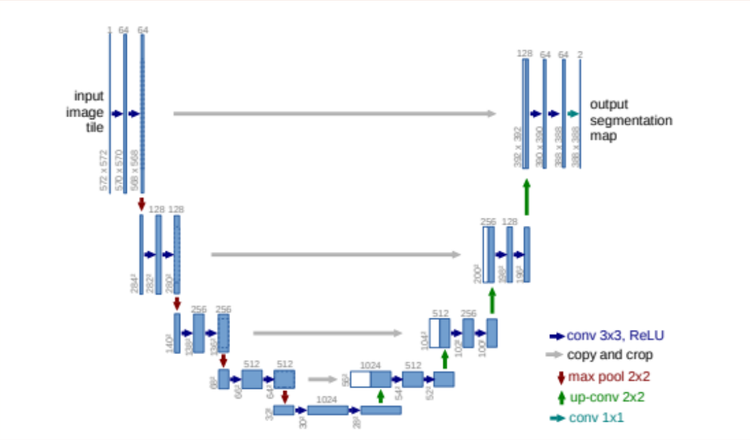
แนวคิดหลักจากเปเปอร์: โครงสร้าง Encoder-Decoder พร้อม Skip Connections ที่ช่วยเชื่อมโยงข้อมูลระดับพิกเซลตื้นเข้ากับระดับลึก

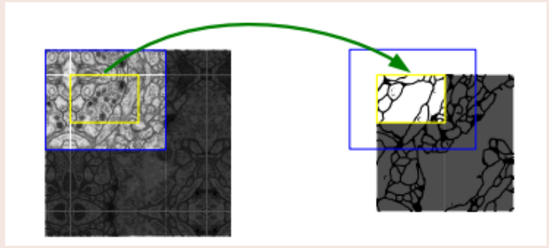


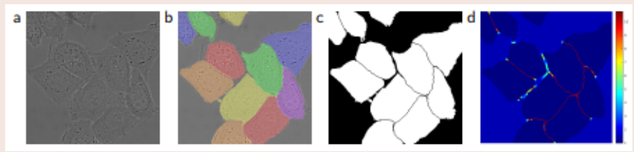
U-Net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, and Thomas Brox

Computer Science Department and BIOSS Centre for Biological Signalling Studies, University of Freiburg, Germany
ronneber@informatik.uni-freiburg.de,
WWW home page: http://lmb.informatik.uni-freiburg.de/



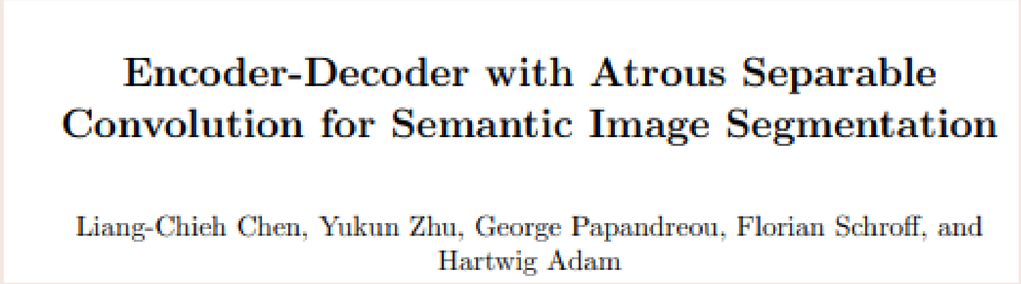




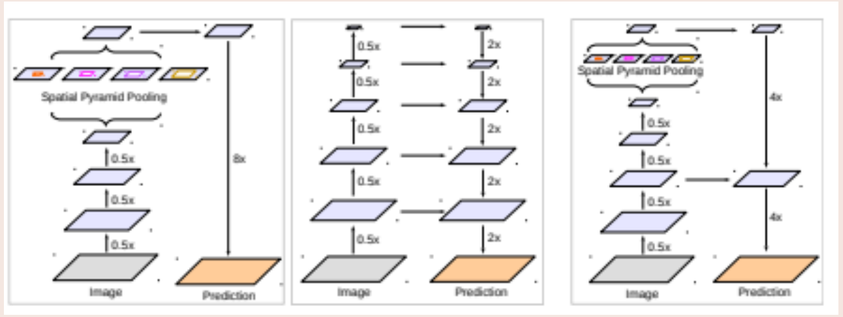
เลือกใช้โครงสร้าง U-Net ตามแนวคิดของ Ronneberger et al. (2015) เนื่องจากมีจุดเด่นด้าน Skip Connections ที่ช่วยถ่ายทอดพิกเซลรายละเอียดขอบเส้นจราจรจากชั้นตื้นสู่ชั้นลึก ทำให้โมเดลสามารถกู้คืนตำแหน่งเส้นเลน (Lane Boundary) ได้อย่างแม่นยำ เหมาะกับงาน Image Segmentation ที่ต้องการความคมชัดของเส้นขอบ

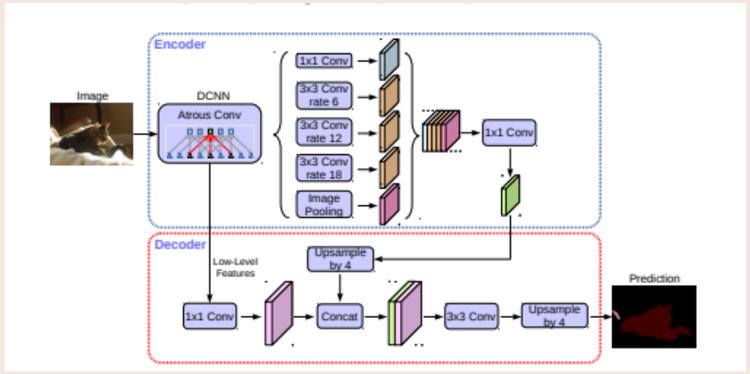
เหตุผลที่ใช้ Unet ในการทดลอง

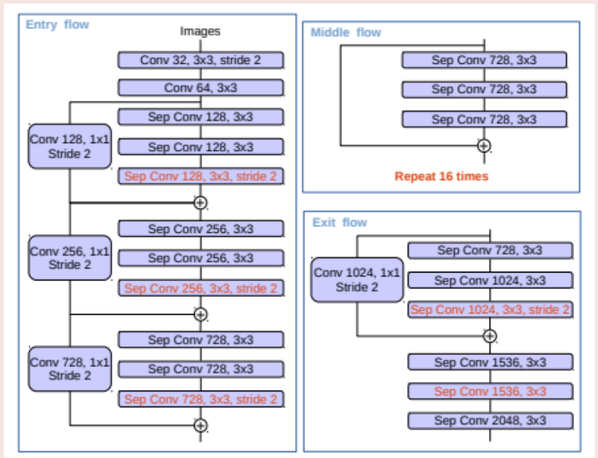
อ้างอิงจากงานวิจัย: Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation


Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation

Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam







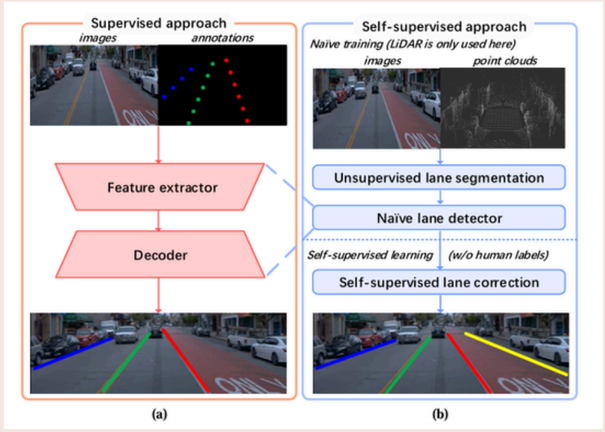
แนวคิดหลักจากเปเปอร์: การใช้โมดูล (ASPP) ที่ใช้ Atrous Convolution ในการขยาย Field of View โดยไม่ลดความละเอียดของภาพ ทำให้โมเดลสามารถเก็บบริบทของภาพในหลายสเกลพร้อมกันได้ ซึ่งช่วยจัดการกับสิ่งรบกวนเฉพาะหน้า เช่น เงาต้นไม้พาดผ่าน หรือแสงสะท้อนบนถนน

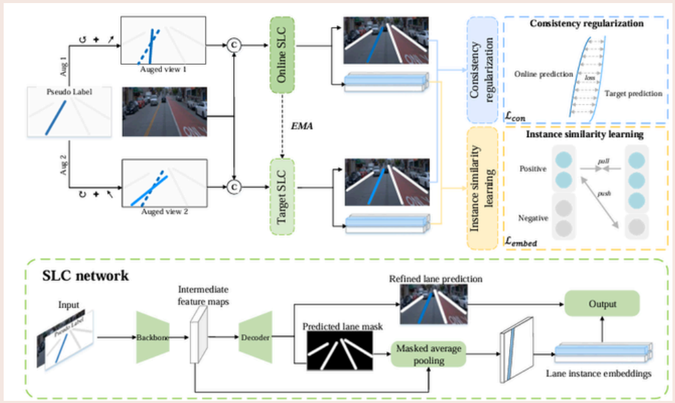
เหตุผลที่ใช้ ssl ในการทดลอง

อ้างอิงจากงานวิจัย: LaneCorrect: Self-supervised Lane Detection (Nie et al., IJCV 2025 / arXiv:2404.14671)

LaneCorrect: Self-supervised Lane Detection

Ming Nie[1] · Xinyue Cai[2] · Hang Xu2 · Li Zhang[1]





โจทย์ปัญหาหลัก ของเปเปอร์:

- งานตรวจจับเส้นถนนแบบดั้งเดิมต้องพึ่งพา Manual Label (แรงงานคนวาดเส้น) จำนวนมหาศาลมักมีสัญญาณรบกวน (Noise) เช่น เส้นขาด หรือตำแหน่งคลาดเคลื่อน และโมเดลเกิดปัญหา Domain Gap เมื่อย้ายพื้นที่ใช้งาน

การทดลอง

Train CULane
Test data set Thai

Train Thai
Test data set Thai

Train Fine-tune
Test data set Thai

Joint Training
Test data set Thai

การทดลอง

Train CULane

ผลลัพธ์การทดลอง ที่ลองเทสถนนในประเทศไทย ไม่เคยเห็นภาพถนนไทยมาก่อน

โมเดล (Architecture + Backbone, Weights)	IoU (%)	Precision (%)	Recall (%)	F1-Score (%)
U-Net (ResNet-50, ImageNet)	39.03%	51.05%	63.72%	54.33%
U-Net (ResNet-50, SSL)	36.86%	47.17%	66.98%	52.48%
U-Net (ResNet-34, ImageNet)	33.99%	42.79%	65.33%	49.23%
DeepLabV3+ (ResNet-50, SSL)	31.16%	40.25%	63.38%	47.52%
DeepLabV3+ (ResNet-50, ImageNet)	31.43%	39.83%	61.59%	46.48%
DeepLabV3+ (ResNet-34, ImageNet)	31.28%	40.65%	61.46%	46.15%

เมื่อนำโมเดลจาก CULane มาทดสอบกับถนนไทยตรงๆ โดยไม่เทรนเพิ่ม จะเห็นปัญหา Domain Gap ทันทีครับ ทุกโมเดลทำ IoU ได้ไม่ถึง 40% โดยเฉพาะค่า Precision ตกลงมาเหลือเพียง 40% เนื่องจากโมเดลติดนิสัยวาดเลนข้างเคียงแบบจีนออกมาแต่เนื่องจากค่า Recall ยังสูงเกิน 60% จึงพิสูจน์ว่าโมเดลตรวจจับเส้นได้ขาดเพียงการปรับจูนให้เข้ากับบริบทถนนไทย และในขั้นนี้ U-Net ได้คะแนนนำ DeepLabV3+ เพราะมุมมองแคบกว่าจึงวาดเส้นหลอกน้อยกว่าครับ

การทดลอง

Train Thai

ผลลัพธ์การทดลอง ที่ลองเทสถนนในประเทศไทย ไม่เคยเห็นภาพถนนไทยมาก่อน

โมเดล (Architecture + Backbone, Weights)	IoU (%)	Precision (%)	Recall (%)	F1-Score (%)
U-Net (ResNet-50, SSL)	51.78%	65.20%	68.96%	66.57%
U-Net (ResNet-34, ImageNet)	50.46%	65.29%	66.35%	65.28%
DeepLabV3+ (ResNet-34, ImageNet)	49.84%	62.16%	68.95%	65.12%
DeepLabV3+ (ResNet-50, ImageNet)	48.97%	59.41%	71.05%	64.24%
DeepLabV3+ (ResNet-50, SSL)	48.90%	61.38%	67.60%	64.08%
U-Net (ResNet-50, ImageNet)	48.61%	62.29%	66.65%	63.88%

ถ้าเราลองเทรนด้วยข้อมูลภาพถนนของไทยล้วนๆ 749 ภาพ โดยไม่พึ่งพาชุดข้อมูล CULane เลย จะเห็นว่าคะแนนจะติดเพดานอยู่แค่ช่วง 48% ถึง 51% เท่านั้นครับ โดยโมเดลที่ดีที่สุดอย่าง U-Net (ResNet-50 SSL) ทำค่า IoU ได้สูงสุดเพียง 51.78% ซึ่งสาเหตุหลักมาจากปัญหา Data Bottleneck เพราะจำนวนข้อมูลที่มีจำกัดเกินไปครับ

การทดลอง

Train Fine-tune

ผลลัพการทดลอง Fine-tuneแล้วเทสจากถนนไทย

โมเดล (Architecture + Backbone, Weights)	IoU (%)	Precision (%)	Recall (%)	F1-Score (%)
DeepLabV3+ (ResNet-50, ImageNet)	50.63%	66.34%	68.14%	67.23%
DeepLabV3+ (ResNet-34, ImageNet)	50.54%	65.44%	68.95%	67.15%
U-Net (ResNet-34, ImageNet)	50.50%	64.29%	70.19%	67.11%
U-Net (ResNet-50, ImageNet)	48.45%	63.61%	67.02%	65.27%
U-Net (ResNet-50, SSL)	47.83%	66.82%	75.41%	64.71%
DeepLabV3+ (ResNet-50, SSL)	46.67%	65.33%	62.03%	63.64%

ส่วนผลการทดลองแบบ Train Fine-tune จะเห็นว่าโมเดล DeepLabV3+ (ResNet-50, ImageNet) ทำผลงานได้ดีที่สุด โดยมีค่า IoU สูงสุดอยู่ที่ 50.63% และ F1-Score อยู่ที่ 67.23% ซึ่งกลุ่มสถาปัตยกรรมของ DeepLabV3+ และ U-Net ที่ใช้ Pretrained Weights จาก ImageNet จะทำคะแนนเกาะกลุ่มกันได้ดีกว่าการใช้ SSL ในชุดการทดลองนี้ครับ

การทดลอง

Joint Training

ผลลัพการทดลอง Joint Training แล้วเทสจากถนนไทย

โมเดล (Architecture + Backbone, Weights)	IoU (%)	Precision (%)	Recall (%)	F1-Score (%)
DeepLabV3+ (ResNet-50, ImageNet)	55.12%	71.04%	69.50%	68.84%
DeepLabV3+ (ResNet-34, ImageNet)	54.00%	70.96%	68.27%	67.91%
DeepLabV3+ (ResNet-50, SSL)	53.99%	68.50%	68.78%	67.26%
U-Net (ResNet-50, ImageNet)	52.71%	70.06%	66.86%	66.51%
U-Net (ResNet-34, ImageNet)	51.25%	67.92%	65.46%	64.77%
U-Net (ResNet-50, SSL)	50.51%	66.98%	63.70%	63.47%

จากผลการทดลองหลังทำ Joint Training จะเห็นว่า DeepLabV3+ ทำผลงานได้เหนือกว่า U-Net ทุกโมเดล โดยเฉพาะ DeepLabV3+ (ResNet-50) ที่ทำคะแนน F1-Score สูงสุดถึง 68.84% และ Precision 71.04% เนื่องจากมีโมดูล ASPP ที่ช่วยจับบริบทและจัดการสิ่งรบกวนบนถนนไทยได้ดีกว่าครับ

สรุปผล

จากการเปรียบเทียบกระบวนการพัฒนาโมเดลทั้งหมด บทสรุปที่ได้คือ DeepLabV3+ ResNet-50
จากวิธี Joint Training คือโมเดลที่สมบูรณ์ที่สุด (IoU 55.12%)
และเป็นโมเดลไฟนอลที่เราเลือกนำมาสร้าง Binary Mask ส่งต่อให้พาร์ท Image Processingใช้งานจริง



DeepLabV3+ (ResNet-50, ImageNet) จากการเทรนแบบ Joint Training

(IoU 55.12% | Precision 71.04% | F1 68.84%)

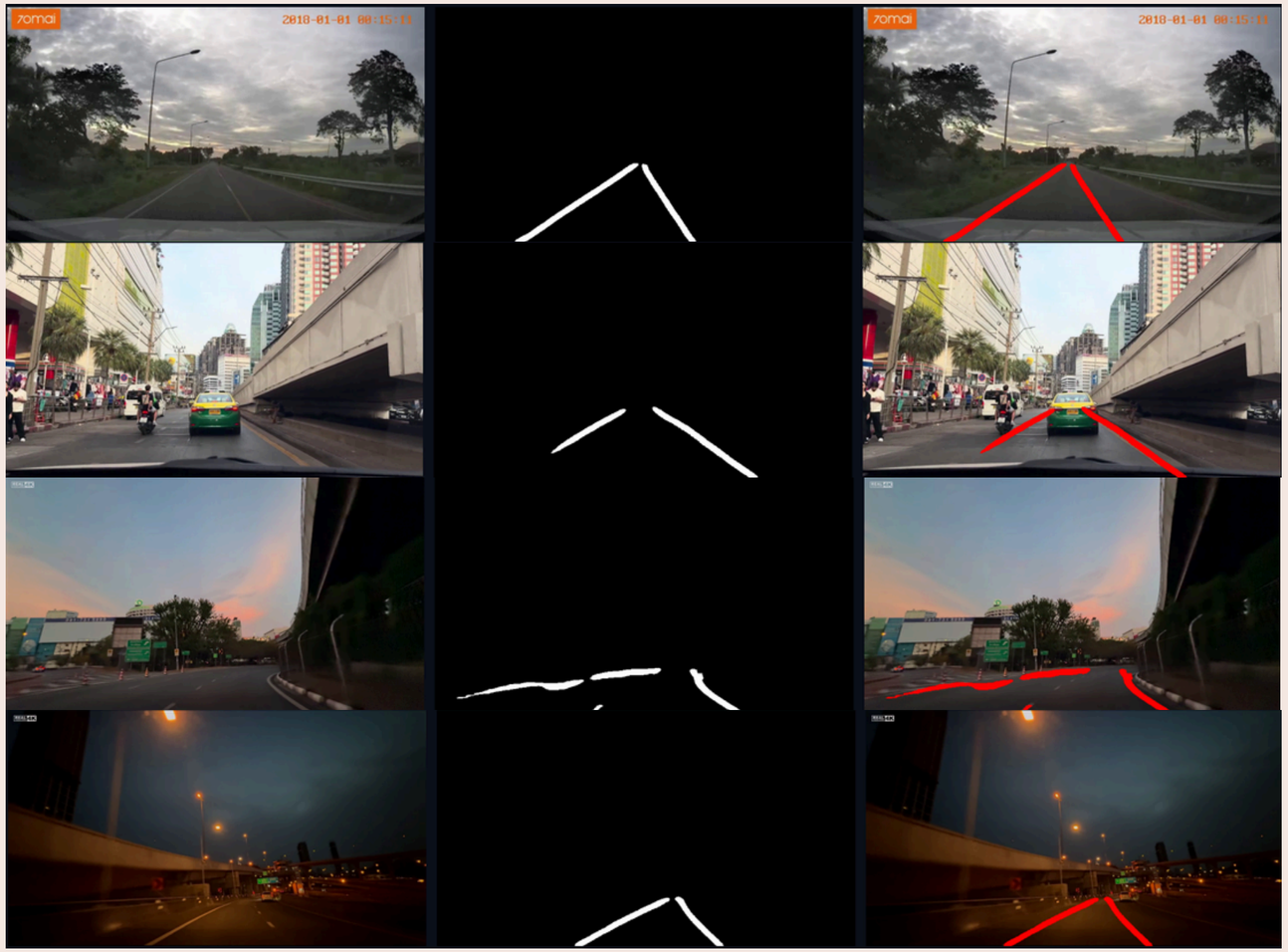
สรุปผล

Model : DeeplabV3+ (Resnet50 , pre-train = Imagenet) เทสกับ Category Thai

หมวดหมู่สภาพถนน (Category)	จำนวนภาพ (N)	IoU (%)	Precision (%)	Recall (%)	F1-Score (%)
Normal (ทางตรงปกติ)	157	61.62%	73.46%	77.89%	75.35%
Curve (ทางโค้ง)	17	28.42%	62.04%	34.30%	42.19%
Night (เวลากลางคืน)	17	21.80%	57.68%	27.23%	35.43%
รวมทั้งหมด (ALL)	191	55.12%	71.04%	69.50%	68.84%

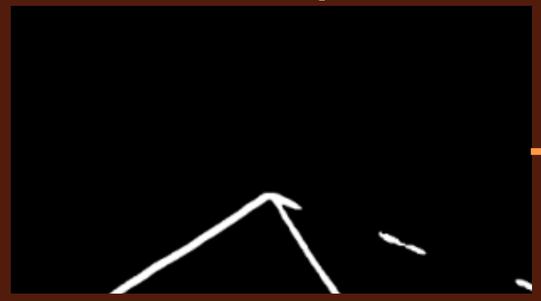
เมื่อทดสอบแยกตามสภาพถนนจริงของไทย พบว่าจุดที่มีความท้าทายคือทางโค้งและเวลากลางคืน
เนื่องจากมีจำนวนภาพในชุดข้อมูลน้อย จึงทำให้ความแม่นยำต่ำกว่าทางตรงที่มีภาพจำนวนมาก
จึงสันนิษฐานได้ว่าหากเก็บภาพทางโค้งและเวลากลางคืนเพิ่มเติม จะช่วยยกระดับความแม่นยำของโมเดลให้สูงขึ้นได้

ผลลัพธ์ ภาพ masks

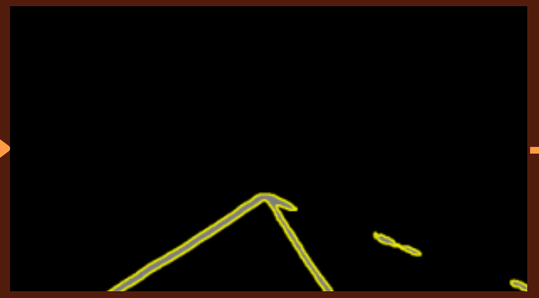


3. การประมวลผลภาพ

mask ที่ได้จากการทำนาย
ของโมเดล DeepLabV3+

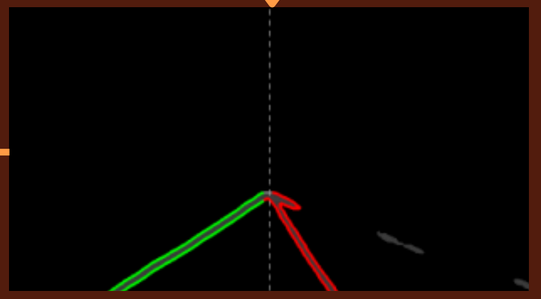


contour

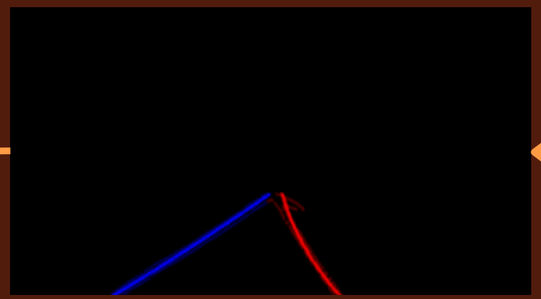


กรองจุดรบกวน

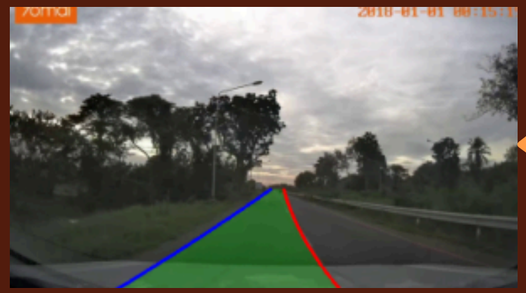




หาเส้นเลนซ้ายและขวา



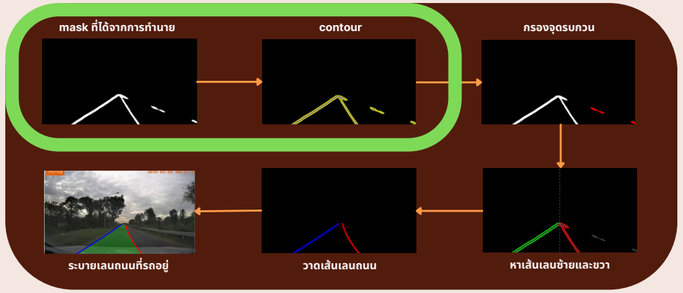
วาดเส้นเลนถนน



ระบายเลนถนนที่รถอยู่

Library: OpenCV (หา Contour, Centroid, วาดและระบายเลน) · NumPy (ฟิตเส้นโค้งพหุนาม) · Matplotlib (แสดงผล)

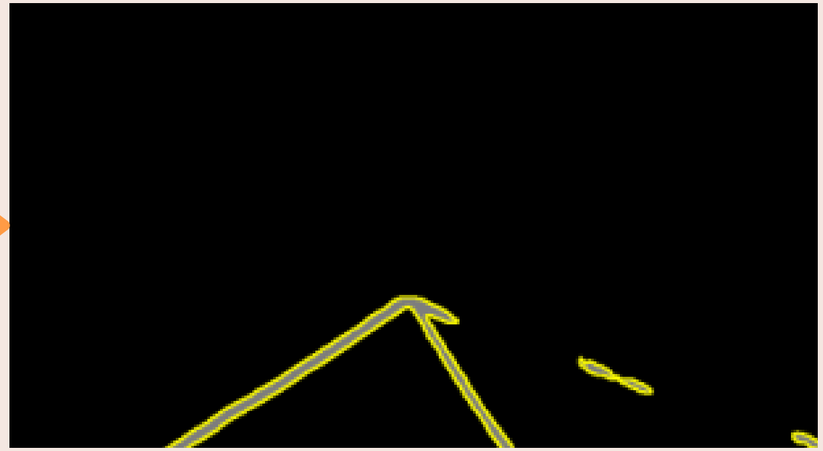
3.1 การ contour



mask ที่ได้จากการทำนาย

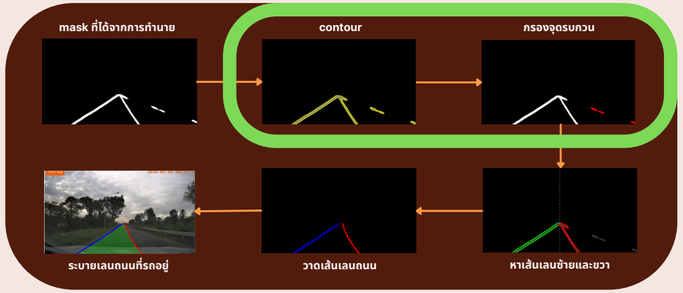


contour

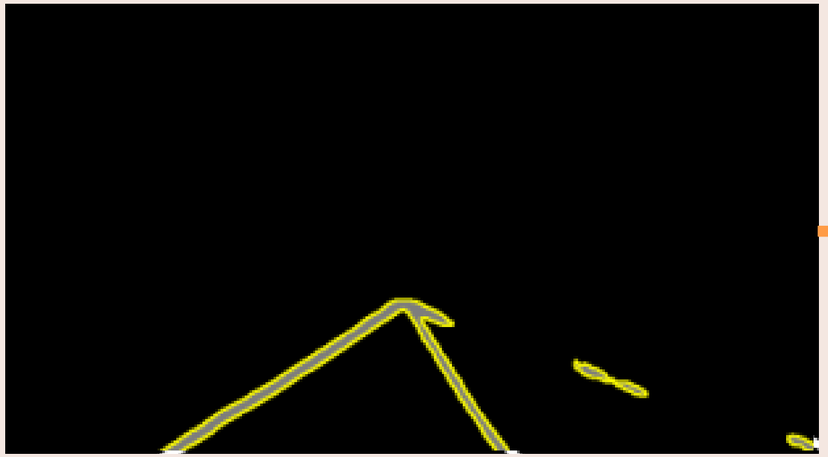


หากลุ่มพิกเซลสีขาวที่ติดกันใน Mask แต่ละกลุ่มคือ 1 Contour (เส้นสีเหลือง) โดยใช้ cv2.findContours เก็บเฉพาะขอบนอก (RETR_EXTERNAL) ได้พิกัดจุดบนขอบของแต่ละก้อน เพื่อแยกเส้นเลนแต่ละเส้นออกจากกัน และวัดพื้นที่หรือหาตำแหน่งของแต่ละก้อน

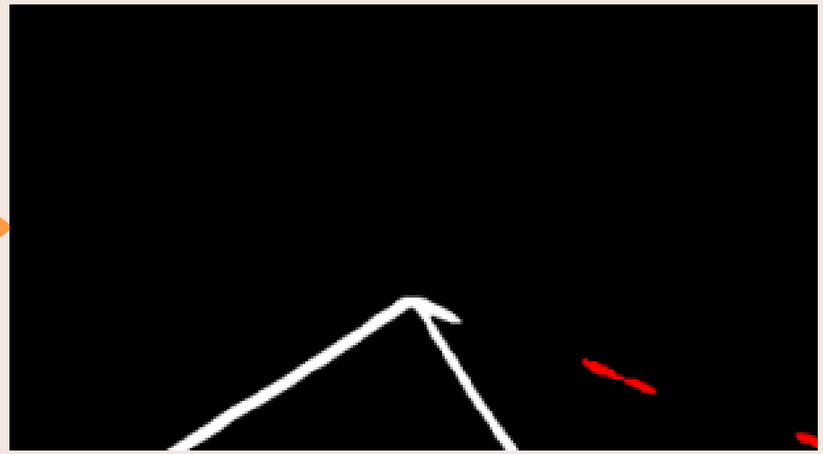
3.2 กรอง Noise



contour

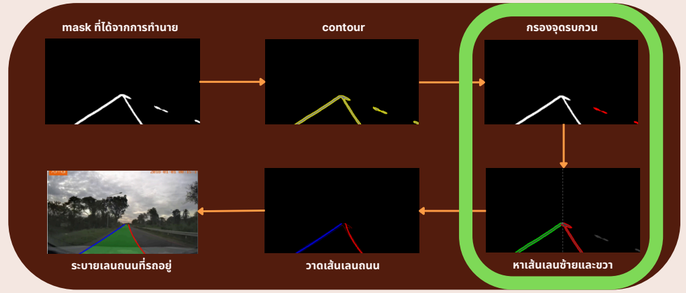


กรองจุดรบกวน

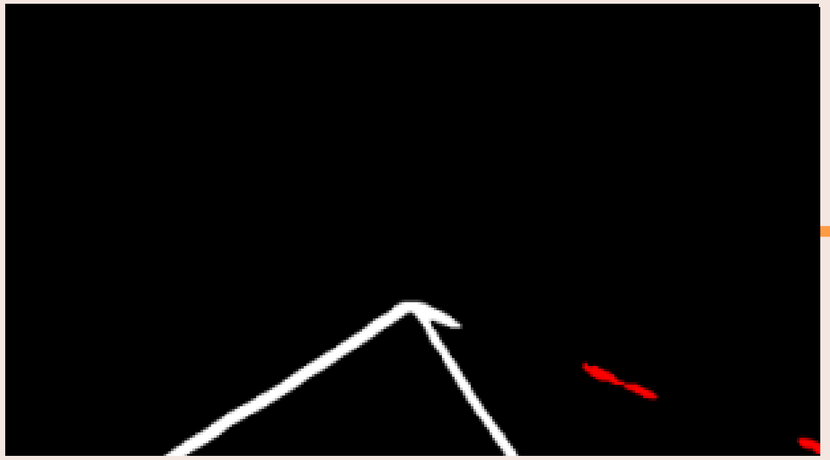


กรอง Contour ขนาดเล็กที่ไม่ใช่เส้นเลน (สีแดง) เหลือเฉพาะเส้นเลน (สีขาว) โดยคำนวณพื้นที่ของแต่ละ Contour ทิ้งก้อนที่เล็กกว่า 500 พิกเซล เลือกใช้ค่า 500 เพราะ เส้นเลนมีพื้นที่ประมาณ 1,000 พิกเซลขึ้นไป จุดรบกวนเกือบทั้งหมดเล็กกว่า 500 ทดลองเทียบกับ 1,000 และ 2,000 แล้ว ค่า 500 วาดพื้นที่เลนได้มากที่สุด

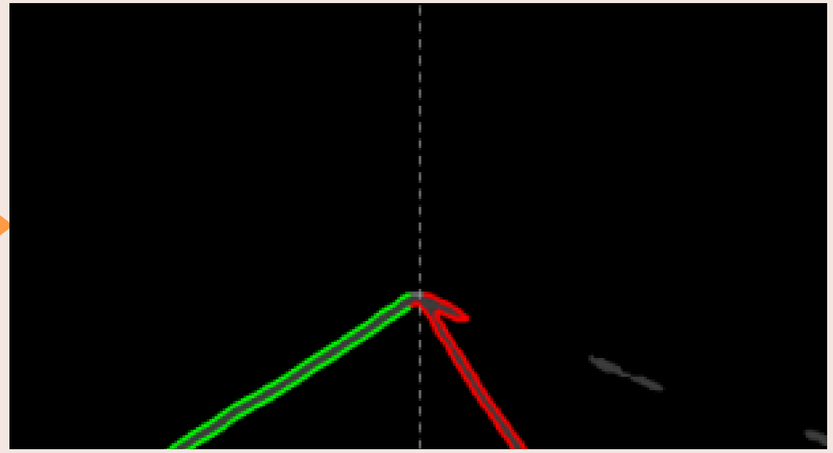
3.3 การแยกเส้นเลนซ้ายและขวาจากรูปร่าง



กรองจุดรบกวน

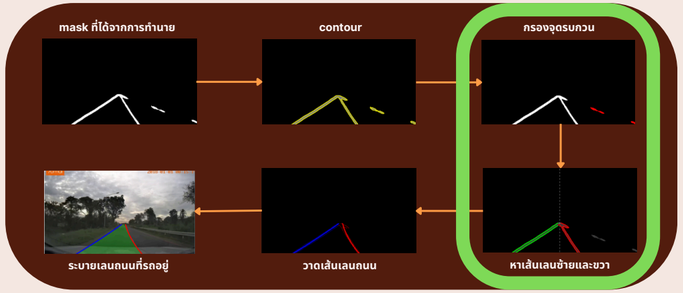


หาเส้นเลนซ้ายและขวา

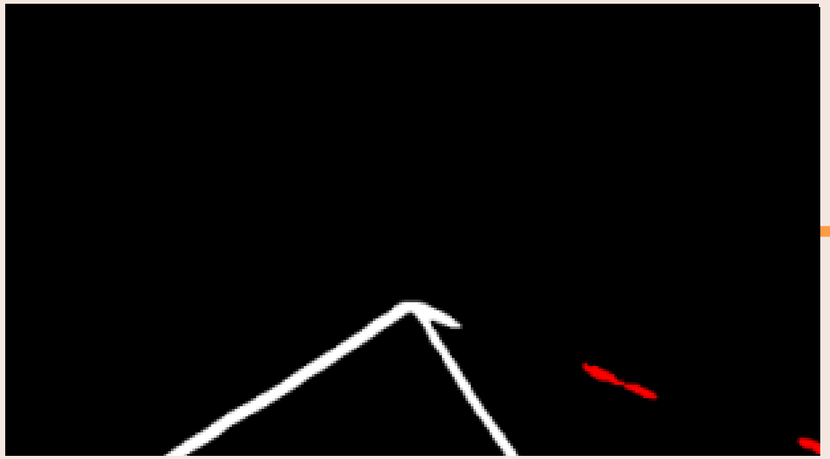


เมื่อได้ contour มาแล้วเราจะทำการหาเส้นเลนด้านซ้ายและเส้นเลนด้านขวา โดยเราจะแบ่งเป็น 2 กรณี เนื่องจาก ภาพ mask ที่ทำนายได้ บางภาพมีเส้นเลนซ้ายและเลนขวาที่มีจุดตัดกันทำให้ contour นับ mask นั้นเป็นเส้นเดียว

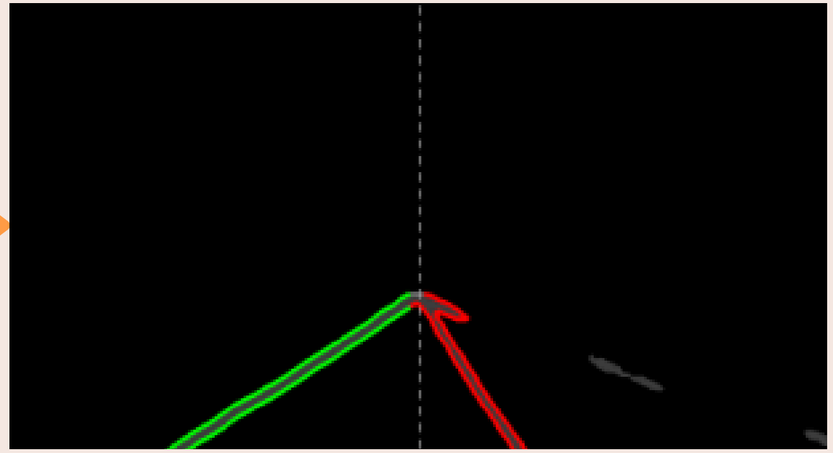
3.3 การแยกเส้นเลนซ้ายและขวาจากรูปร่าง



กรองจุดรบกวน

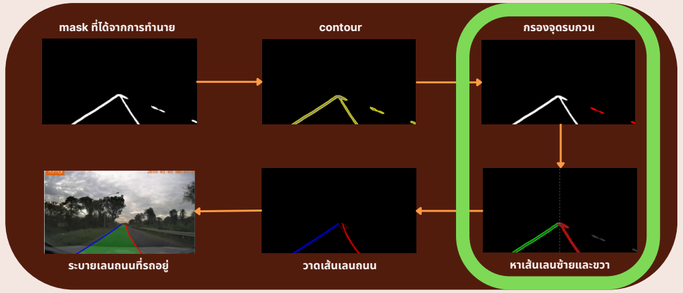


หาเส้นเลนซ้ายและขวา

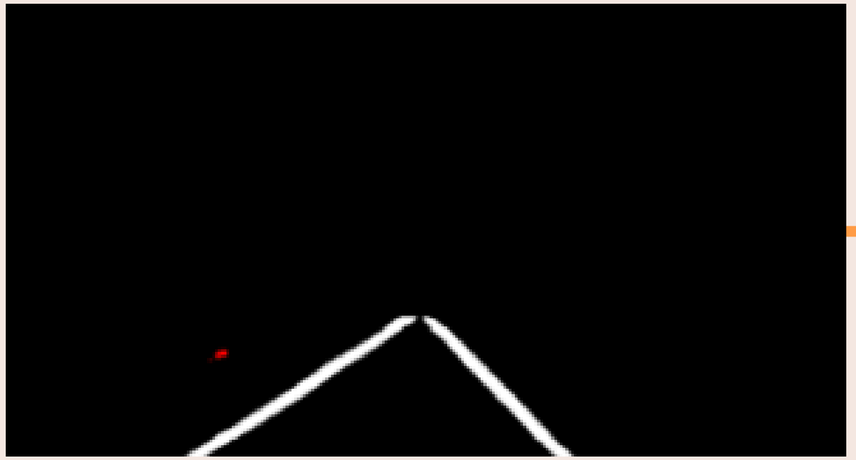


กรณีที่ 1 เส้นติดกัน: Mask บางภาพเส้นซ้ายและขวาชนกันเป็นก้อนเดียว ตรวจจากจุดยอดสูงสุดของก้อน ถ้าทั้งสองข้างกว้างเกิน 20 พิกเซล ถือว่าเป็นสองเส้นติดกัน จึงแบ่งที่จุดยอดเป็นเส้นซ้ายและเส้นขวา

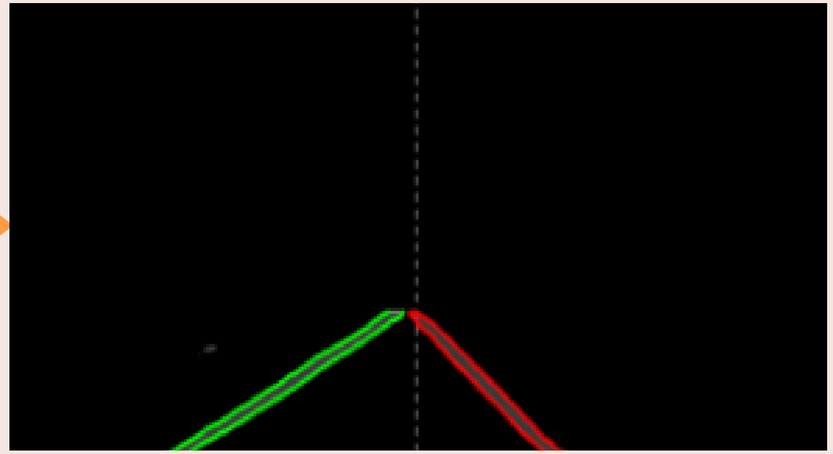
3.3 การแยกเส้นเลนซ้ายและขวาจากรูปร่าง



กรองจุดรบกวน

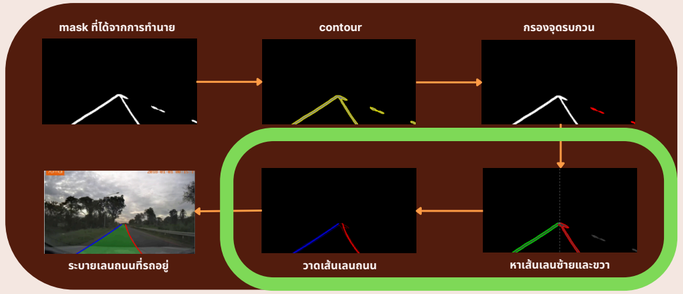


หาเส้นเลนซ้ายและขวา

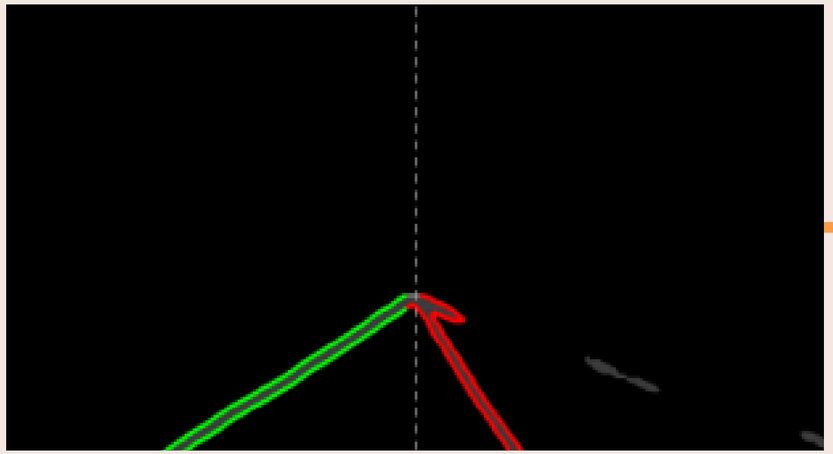


กรณีที่ 2 เส้นแยกกัน: หาจุดศูนย์กลางมวล (Centroid) ของแต่ละก้อน เทียบกับแกนกึ่งกลางภาพ x = 400 (เส้นประ) น้อยกว่าเป็นเส้นซ้าย มากกว่าเป็นเส้นขวา
เลือกฝั่งละ 1 เส้น: ถ้าฝั่งเดียวกันมีหลายก้อน เลือกก้อนที่ใกล้แกนกึ่งกลางที่สุด ซึ่งเป็นขอบของเลนที่รถวิ่งอยู่ (ก้อนสีเทาคือเส้นของเลนอื่น ไม่ถูกเลือก)

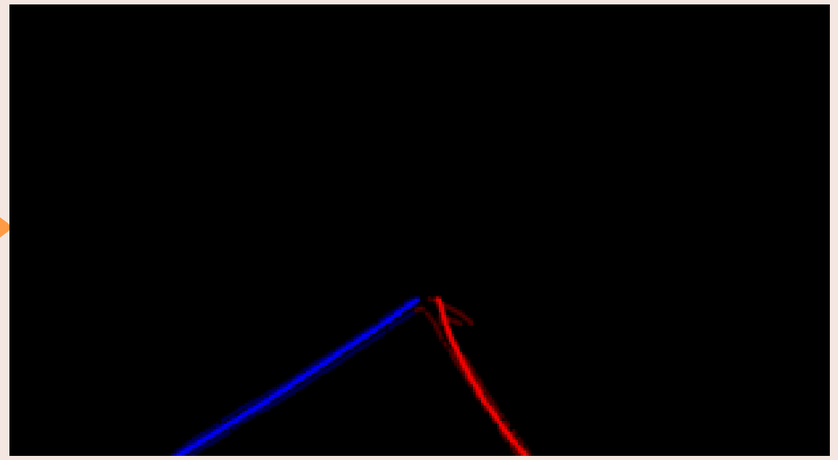
3.4 การประมาณเส้นโค้งด้วยการถดถอยพหุนาม



หาเส้นเลนซ้ายและขวา

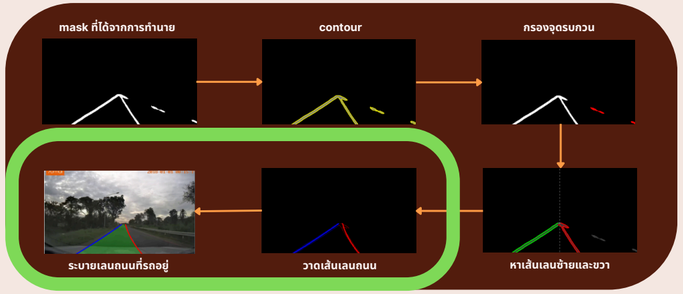


วาดเส้นเลนถนน

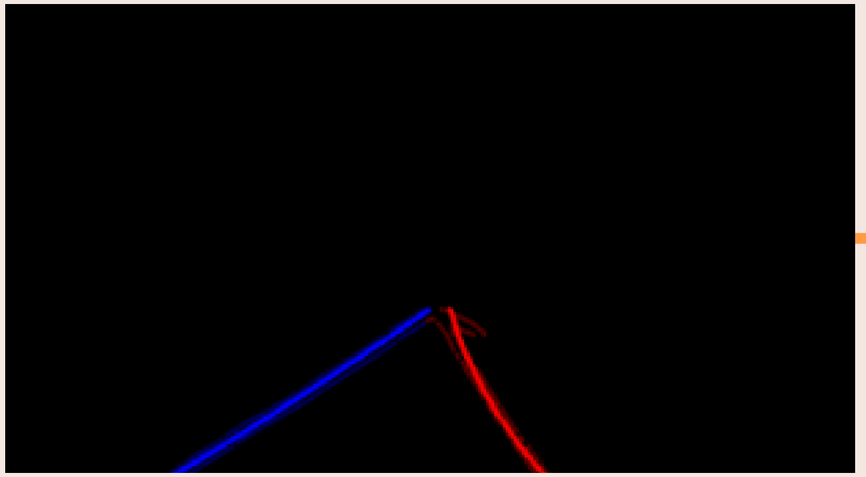


แปลงจุดของแต่ละเส้นให้เป็นเส้นโค้งที่เรียบและต่อเนื่อง (ซ้ายสีน้ำเงิน ขวาสีแดง) โดยใช้การถดถอยพหุนามดีกรีสอง $x = ay^2 + by + c$ เลือกดีกรีสองเพราะรองรับทางโค้ง โดยไม่บิดตามจุดรบกวน ใช้ x เป็นฟังก์ชันของ y เพราะเส้นเลนเกือบตั้งตรง

3.5 การระบายพื้นที่เส้นเลนถนน



วาดเส้นเลนถนน



ระบายเลนถนนที่รถอยู่



ระบายพื้นที่ระหว่างเส้นซ้ายและเส้นขวาบนภาพจริง เพื่อแสดงเลนที่รถกำลังขับอยู่ โดยคำนวณจุดบนเส้นโค้ง 100 จุดต่อเส้น ในช่วงความสูงที่ทั้งสองเส้นมีร่วมกัน ต่อเป็นรูปหลายเหลี่ยมสีเขียว ผสมกับภาพจริงสัดส่วน 30 : 70 เพื่อให้แสดงเป็นพื้นที่โปร่งแสงที่ยังเห็นถนนจริงข้างใต้ ถ้าหาเส้นได้ไม่ครบสองฝั่ง จะไม่ระบายเลนถนน

ผลการวาดพื้นที่เลนถนน

	ทางตรง (157 ภาพ)	ทางโค้ง (17 ภาพ)	กลางคืน (17 ภาพ)	รวม (191 ภาพ)
ค่าในรูปเล่ม	147 (93.6%)	11 (64.7%)	14 (82.4%)	172 (90.1%)
ค่าอัปเดตที่ถูกต้อง	**147 (93.6%)**	**12 (70.6%)**	**16 (94.1%)**	**175 (91.6%)**

วาดพื้นที่เลนได้ = พบเส้นเลนครบสองฝั่งและวาดพื้นที่เลนได้ (ยังไม่ได้ตรวจความถูกต้อง)

*** ตัวเลขในเล่ม (172 ภาพ) ได้จากโค้ดเวอร์ชันก่อน ที่รวมทุก Contour ในแต่ละฝั่ง ซึ่งไม่ตรงกับหัวข้อ 5.8**

ข้อค้นพบ

- วาดพื้นที่เลนได้ 175 จาก 191 ภาพ (ร้อยละ 91.6) ภาพที่เหลือพบเส้นเลนได้ฝั่งเดียว ไม่มีภาพที่หาเส้นไม่พบ แต่เมื่อตรวจด้วยตา พบภาพที่วาดเลนถนนได้ถูกต้อง 117 ภาพ

ผลการตรวจความถูกต้องด้วยตา

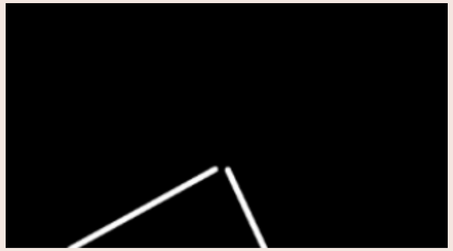
ผลตรวจ	ทางตรง (157)	ทางโค้ง (17)	กลางคืน (17)	รวม (191)
วาดพื้นที่เลนได้ (both)	147 (93.6%)	12 (70.6%)	16 (94.1%)	175 (91.6%)
✓ ถูกต้อง	**109 (69.4%)**	**0 (0.0%)**	**8 (47.1%)**	**117 (61.3%)**
~ ถูกช่อง แต่เพี้ยน	32 (20.4%)	0 (0.0%)	7 (41.2%)	39 (20.4%)
✗ ผิด	6 (3.8%)	12 (70.6%)	1 (5.9%)	19 (9.9%)
วาดไม่ได้ (one)	10 (6.4%)	5 (29.4%)	1 (5.9%)	16 (8.4%)

เทียบผลระบายเลนกับภาพต้นฉบับและ Ground Truth ทุกภาพ (191 ภาพ)

ข้อค้นพบ

- วาดพื้นที่เลนได้ 175 ภาพแต่ถูกต้องจริง 117 ภาพ (ร้อยละ 61.3)
- ผลทางตรงทำได้ดีที่สุด ถูกต้อง 109 ภาพ จาก 157 ภาพ (ร้อยละ 69.4)
- ทางโค้ง ภาพที่วาดได้ทั้งหมดยังไม่ถูกต้อง

ตัวอย่างผลลัพธ์

	ภาพต้นฉบับ	Ground Truth	Mask ทำนาย	ผลระบายเลนถนน
ทางตรงที่ระบายเลนได้ถูกต้อง ✅				

กรองจุดรบกวนออก แบ่งเส้นที่ติดกันที่จุดยอดเป็นเส้นซ้ายและขวา แล้วระบายเลนได้ถูกต้อง

ถนนกลางคืนที่ระบายเลนได้ถูกต้อง ✅				

เส้นกลางถนนจาง แต่โมเดลตรวจพบเส้นกลางถนน จึงระบายเฉพาะช่องที่รถวิ่งอยู่ได้ถูกต้อง

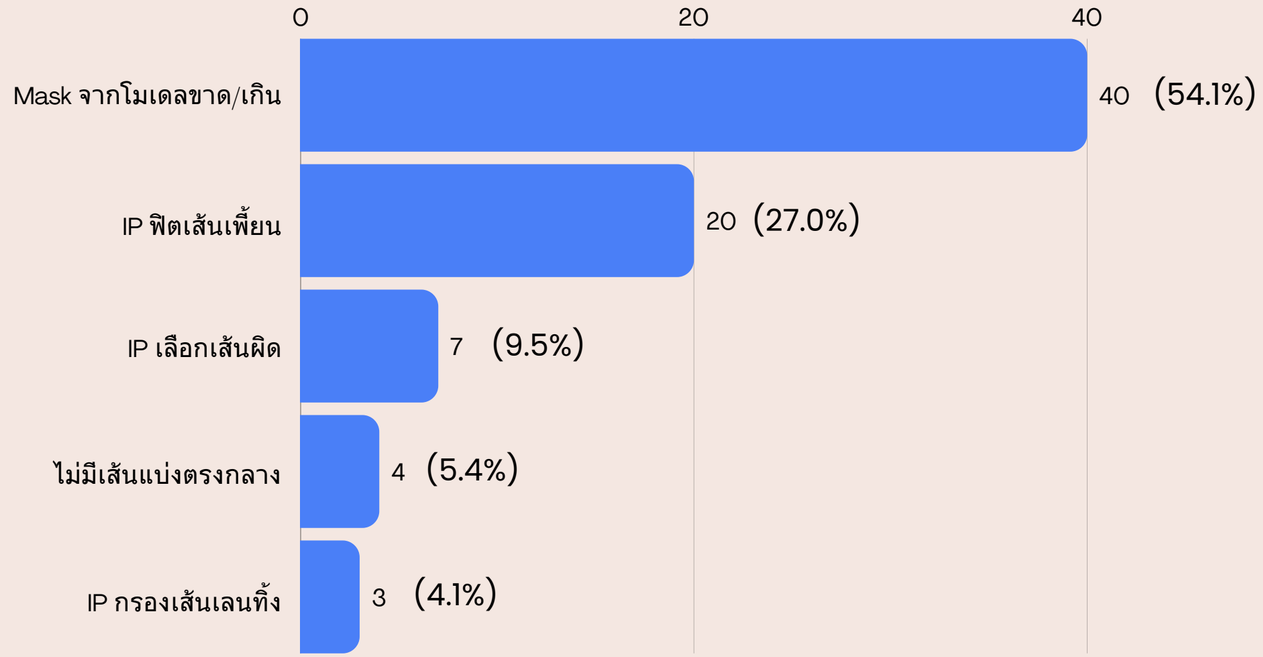
ทางโค้งระบายเลนผิด ❌				

Mask ไม่มีเส้นขวา และปลายเส้นซ้ายขาดเป็นท่อนแนวนอน ระบบเข้าใจว่าท่อนนั้นเป็นเส้นสองเส้นติดกัน จึงแบ่งเป็นเส้นซ้ายและขวาจากท่อนเดียวกัน พื้นที่ที่ระบายจึงเล็กและผิดตำแหน่ง

สาเหตุของภาพที่ไม่ถูกต้อง

ข้อค้นพบ

- สาเหตุหลักมาจาก Mask ของโมเดล ร้อยละ 54.1 ของภาพที่ไม่ถูกต้อง
- ขั้นตอนการประมวลผลภาพมีส่วนร้อยละ 40.5 (ฟิตเส้นเพี้ยน 20 ภาพ เลือกเส้นผิด 7 ภาพ กรองเส้นทิ้ง 3 ภาพ)
- ทางโค้งพลาดจาก Mask ขาดเส้นเลนถนนมากที่สุด (10 จาก 17 ภาพ)



นับจากภาพที่ไม่ถูกต้องและภาพที่วาดไม่ได้ รวม 74 ภาพ (ร้อยละคิดจาก 74 ภาพ)

ผลการดำเนินงาน

ด้านโมเดล

- โมเดลที่ดีที่สุดคือ DeepLabV3+ (ResNet-50, ImageNet) ที่ฝึกแบบ Joint Training (รวมชุดข้อมูล CULane และถนนไทย)
- ได้ IoU 55.12%, Precision 71.04% และ F1-Score 68.84% สูงที่สุดในทุกการทดลอง
- โมดูล ASPP ช่วยจับบริบทของภาพได้หลายขนาด จึงรับมือสิ่งรบกวนบนถนนไทยได้ดีกว่า U-Net
- ใช้ BCE + Dice Loss ช่วยให้โมเดลเรียนรู้เส้นเลนที่มีพิกเซลน้อยกว่าพื้นหลังมาก

ผลการดำเนินงาน

ด้านการประมวลผลภาพ

- วาดพื้นที่เลนได้ 175 จาก 191 ภาพ (ร้อยละ 91.6) และพบเส้นเลนอย่างน้อยหนึ่งฝั่งในทุกภาพ
- เมื่อตรวจด้วยตาเทียบกับภาพจริงและ Ground Truth ระบายถูกต้อง 117 ภาพ (ร้อยละ 61.3) ทางตรงดีที่สุด 109 จาก 157 ภาพ (ร้อยละ 69.4) ส่วนทางโค้งยังระบายไม่ถูกต้อง
- ภาพที่ไม่ถูกต้อง 74 ภาพ มีสาเหตุหลักจาก Mask ของโมเดล (ร้อยละ 54.1) รองลงมาคือขั้นตอนการประมวลผลภาพ (ร้อยละ 40.5)

ภาพรวม: คุณภาพของ Mask จากโมเดล เป็นปัจจัยหลักที่กำหนดผลของระบบทั้งหมด

ข้อจำกัด

ด้านโมเดล

- ทางโค้งและกลางคืนยังทำได้ต่ำกว่าทางตรงชัดเจน (IoU ทางโค้ง 28.42% และกลางคืน 21.80% เทียบกับทางตรง 61.62%)
- ข้อมูลทางโค้งและกลางคืนมีน้อย (หมวดละ 17 ภาพในชุดทดสอบ) และประสิทธิภาพยังไม่ถึงจุดอิ่มตัว
- ข้อมูลยังไม่ครอบคลุมสภาวะที่ท้าทาย เช่น แสงน้อย แสงจ้า และเส้นที่ลบเลือน
- ถนนที่ไม่มีเส้นแบ่งตรงกลาง หรือเส้นจางจนมองไม่เห็น label ได้เฉพาะเส้นขอบถนน จึงครอบสองช่องจราจร

ข้อจำกัด

ด้านการประมวลผลภาพ

- ผลขึ้นกับคุณภาพของ Mask ถ้า Mask ขาดเส้น ระบบจะระบายผิดหรือวาดไม่ได้ (ทางโค้งวาดไม่ได้ 5 ภาพ และวาดได้แต่ไม่ถูกต้อง 12 ภาพ จาก 17 ภาพ)
- การเลือกเส้นที่ใกล้กึ่งกลางภาพ อาจเลือกชิ้นส่วนสั้นที่ไม่ใช่เส้นเลนจริง
- อาศัยสมมติฐานว่ากล้องติดกึ่งกลางหน้ารถ ถ้าเส้นเลยกึ่งกลางภาพ อาจจัดผิดฝั่ง
- ฟิตเส้นจากจุดบนขอบ Contour เส้นจึงอาจเพี้ยนหรือปลายงอ (20 ภาพ)
- เกณฑ์ 500 พิกเซลอาจกรองท่อนเส้นเลนจริงที่ขาดเป็นชิ้นเล็กทิ้ง (3 ภาพ)
- ประมวลผลทีละภาพ ยังไม่ใช้ความต่อเนื่องระหว่างเฟรมของวิดีโอ
- ตรวจความถูกต้องด้วยตา ยังไม่มีค่าวัดเชิงตัวเลข

ข้อเสนอแนะ

ด้านข้อมูลและแบบจำลอง

- เก็บข้อมูลเพิ่มในสภาวะที่ยังทำได้ไม่ดี ได้แก่ ทางโค้ง กลางคืน แสงจ้า และเส้นที่ลบเลือน
- กำหนดแนวทางการ label สำหรับถนนที่ไม่มีเส้นแบ่งช่องจราจร

ด้านการประมวลผลภาพ

- เลือกเส้นโดยดูความยาวของ Contour ร่วมกับตำแหน่ง และรวมท่อนเส้นประที่อยู่แนวเดียวกันก่อนเลือก
- ตัดท่อนที่เกือบแนวนอนออกก่อนแยกเส้นซ้าย–ขวา
- ฟิตเส้นจากพิกเซลทั้งหมดของเส้น แทนจุดบนขอบ Contour
- วัดผลเป็นตัวเลข เช่น IoU ของพื้นที่เลนเทียบกับ Ground Truth

แนวทางในอนาคต

- ใช้ความต่อเนื่องระหว่างเฟรมของวิดีโอ เช่น ใช้ Temporal Smoothing เฉลี่ยสมการเส้นจากหลายเฟรม
- แปลงโมเดลให้น้ำหนักเบา (TensorFlow Lite หรือ ONNX) เพื่อประมวลผลแบบเรียลไทม์บน Raspberry Pi 5 และพัฒนาเป็นกล้องหน้ารถแจ้งเตือน

Thank You.